A

ABSTRACT REPORT

On

AI POWERED SAST AGENT

Submitted by
Sai Sanjana Gujjari (2170-22-026-028)
Dumpala Pardha Saradhi (2170-22-026-007)

Under the Guidance of Miss Roshini Chinthala

Department of BS-MS (Computer Science)
VISHWA VISHWANI INSTITUTE OF SYSTEMS & MANAGEMENT
AFFLIATED TO OSMANIA UNIVERSITY, HYDERABAD

1. Overview of the Project

The AI Powered SAST Agent is a full-stack web platform that performs Static Application Security Testing (SAST) using Large Language Models (LLMs) as the core scanning engine. Unlike traditional SAST tools that rely on rule-based binaries, this system submits source code files directly to high-performance AI models which identify, classify, and explain security vulnerabilities in context.

Developers submit any GitHub repository URL or ZIP archive through a clean web dashboard. The AiScannerService recursively walks the codebase, sends each file to a multi-provider LLM pipeline (NVIDIA NIM → Google Gemini 1.5 Flash → OpenAI GPT-4o-mini), and returns structured JSON findings with severity level, affected line numbers, CWE/OWASP identifiers, plain-English explanation, proof-of-concept attack scenario, and an exact code-level fix. Results are streamed live to the dashboard via Server-Sent Events (SSE) and persisted in PostgreSQL for historical comparison.

2. Existing System

Current SAST solutions such as SonarQube, Semgrep, and Checkmarx are built on curated rule libraries that match known vulnerability patterns against source code. While effective at detecting well-defined issues, these tools suffer from several significant drawbacks:

- High false-positive rates — rule engines flag large volumes of non-exploitable issues, causing alert fatigue among developers.
- Limited context understanding — findings are presented as raw technical identifiers without explaining why the issue is dangerous or how to fix it.
- Rule maintenance burden — every new vulnerability class or framework requires manual rule authoring and library updates.
- CLI-only interfaces — most tools require security expertise to operate and interpret, excluding the majority of development teams.
- No unified platform — scanning, explanation, and remediation tracking are spread across multiple disconnected tools.

3. Proposed System

The AI Powered SAST Agent addresses all existing limitations by using LLMs as the primary scanning engine — eliminating rule libraries entirely. The proposed system provides:

- AI-Native Scanning: The AiScannerService walks the repository and submits each source file to an LLM with a structured security audit prompt, receiving findings as structured JSON without any external binary dependency.

- Multi-LLM Resilience: A prioritised fallback pipeline — NVIDIA NIM (Llama 3.1) for ultra-fast inference, Google Gemini 1.5 Flash as the primary engine, and OpenAI GPT-4o-mini as fallback — ensures maximum availability under load.
- AI Enrichment per Finding: Every vulnerability is enriched with a plain-English explanation, contextual impact assessment, proof-of-concept attack scenario, CWE/OWASP mapping, and a ready-to-apply code fix.
- Real-Time Dashboard: Findings are streamed live via SSE to a React dashboard with severity charts (Recharts), a Monaco Editor code viewer with vulnerable line highlighting, and a full AI Insight Panel.
- Scan History & Comparison: All scans are persisted in PostgreSQL, enabling side-by-side comparison of any two scans to track newly introduced and resolved vulnerabilities over time.
- Secure & Containerised: GitHub OAuth authentication, JWT sessions, Helmet.js security headers, Zod validation, and fully Dockerised deployment via Docker Compose.

4. Software Requirements

- Operating System: Windows 10/11, macOS 12+, or Ubuntu 22.04+
- Runtime: Node.js 20 LTS
- Database: PostgreSQL 16.x
- Cache / Session Store: Redis 7.x
- Frontend: React 18.x, Vite 5.x, Tailwind CSS 3.x, shadcn/ui, Recharts 2.x, Monaco Editor
- Backend: Express.js 4.x, TypeScript 5.x, Prisma ORM 5.x
- Authentication: Passport.js with GitHub OAuth 2.0, Helmet.js, Zod 3.x
- AI Providers: NVIDIA NIM (Llama 3.1), Google Gemini 1.5 Flash API, OpenAI GPT-4o-mini API
- Containerisation: Docker Engine 24+ and Docker Compose
- Web Server: Nginx Alpine (reverse proxy)
- Browser: Chrome 120+ or Firefox 120+

5. Hardware Requirements

- Processor: Minimum dual-core 2.0 GHz (x86-64); Recommended quad-core 3.0 GHz or higher
- RAM: Minimum 8 GB; Recommended 16 GB
- Disk Space: Minimum 20 GB free SSD; Recommended 50 GB SSD
- Network: Stable broadband internet connection (required for LLM API calls and GitHub cloning)
- Docker: Docker Desktop or Docker Engine 24+
- Display: 1280×720 minimum resolution for dashboard usability